Performance Analysis Report

Numerical Integration with Parallel Implementations

HPC — Assignment 1

Serial | OpenMP | CUDA (Design)

Team: 4

Name	BN
1-Goda Saber Goda	5
2-Hamed Essam Hamed	6
3-Mohamed Abdel Moneim	29

1. Introduction

Numerical integration is widely used to approximate definite integrals when analytical solutions are difficult or unavailable. This project presents the design and implementation of a flexible numerical integration tool that evaluates user-defined mathematical expressions over a given interval $[a, b]$.

The system supports three execution models:

- **Serial Implementation**: A baseline sequential approach running on a single CPU core.
- **OpenMP Implementation**: A multi-threaded CPU approach using shared-memory parallelism.
- **CUDA Implementation**: A GPU-based approach designed for large-scale parallel computation (implemented but not executed due to hardware limitations).

Users input the integrand as a string expression (e.g., "x^2 + sin(x) + 2*x"). The expression is parsed into **Reverse Polish Notation (RPN)** using the Shunting-Yard algorithm and evaluated efficiently at runtime.

Objectives

- Validate correctness against known analytical results.
- Measure execution time as the number of intervals $n$increases.
- Evaluate speedup achieved using OpenMP.
- Present and justify the CUDA parallel design

```
hamed@DESKTOP-3JEVOT9:~/integration-project11/src$ ./integrator -h
Usage: ./integrate [options]
Options:
  --function EXPR   Mathematical expression (default: 'x*x')
  --a FLOAT         Lower limit (default: 0)
  --b FLOAT         Upper limit (default: 1)
  --n INT           Number of intervals (default: 1000000)
  --method METHOD   riemann, simpson (default: riemann)
  --impl TYPE       serial, omp, cuda (default: serial)
  --threads INT     OpenMP threads (default: 1)
  --block-size INT  CUDA block size (default: 256)
  --repeats INT     Repetitions for averaging (default: 5)
  --output FILE     Save CSV results (optional)
  --help            Show this help
```

2. Algorithms and Implementation

2.1 Numerical Integration Methods

Midpoint Riemann Rule

$$\int_a^b f(x)dx \approx \frac{b-a}{n}\sum_{i=0}^{n-1} f\left(a+\left(i+\frac{1}{2}\right)\frac{b-a}{n}\right)$$

The interval $[a, b]$is divided into $n$equal subintervals. The function is evaluated at the midpoint of each subinterval.

- $dx = \frac{b-a}{n}$
- Each iteration computes one independent function evaluation

Advantages:

- Simple and efficient
- Highly parallelizable (no dependencies between iterations)

```
if (method == "riemann") {
    for (long long i = 0; i < n; ++i) {
        double x = a + (i + 0.5) * dx;
        sum += eval_rpn(rpn, x);
    }
    return sum * dx;
```

Simpson's Rule

Simpson's rule provides higher accuracy by approximating the function using quadratic interpolation.

It uses weighted sums:

- Odd indices → weight = 4
- Even indices → weight = 2

Advantages:

- More accurate than Riemann
- Still parallelizable (with careful summation)

```
else if (method == "simpson") {
    if (n % 2 != 0) {
        std::cerr << "Warning: Simpson's rule requires even n. Increasing n by 1.\n";
        n++;
        dx = (b - a) / n;
    }
    double sum_odd = 0.0, sum_even = 0.0;
    for (long long i = 1; i < n; i += 2) {
        double x = a + i * dx;
        sum_odd += eval_rpn(rpn, x);
    }
    for (long long i = 2; i < n; i += 2) {
        double x = a + i * dx;
        sum_even += eval_rpn(rpn, x);
    }
    double fa = eval_rpn(rpn, a);
    double fb = eval_rpn(rpn, b);
    return (dx / 3.0) * (fa + fb + 4.0 * sum_odd + 2.0 * sum_even);
}
```

2.2 Expression Parsing — Shunting-Yard Algorithm

Instead of relying on an external library, a custom parser converts the infix expression to Reverse Polish Notation (RPN) using Dijkstra's Shunting-Yard algorithm. The RPN token array is evaluated at runtime using a small stack, on both CPU and GPU.

Supported grammar:

- Binary operators: + - * / ^ (exponentiation)
- Unary functions: sin cos tan exp log
- Variable x, numeric constants, and nested parentheses.

The parser runs once on the host. The resulting `CompiledRPN` object is reused for all n evaluations, eliminating repeated parsing overhead.

```cpp
CompiledRPN compile_expression(const std::string& expr) {
    std::vector<RPNToken> output;
    std::stack<char> ops;

    size_t i = 0;
    while (i < expr.size()) {
        skip_spaces(expr, i);
        if (i >= expr.size()) break;

        char c = expr[i];
        if (std::isdigit(c) || c == '.') {
            double num = parse_number(expr, i);
            output.push_back({TokenType::NUM, num});
            continue;
        }
        else if (c == 'x' || c == 'X') {
            output.push_back({TokenType::VAR, 0.0});
            ++i;
            continue;
        }
        else if (c == '+' || c == '-' || c == '*' || c == '/' || c == '^') {
            while (!ops.empty() && ops.top() != '(' &&
                   ((precedence(ops.top()) > precedence(c)) ||
                    (precedence(ops.top()) == precedence(c) && !right_associative(c)))) {
                output.push_back({static_cast<TokenType>(ops.top()), 0.0});
                ops.pop();
            }
            ops.push(c);
            ++i;
        }
```

```cpp
double eval_rpn(const CompiledRPN& rpn, double x)!! {
    std::stack<double> st;
    for (const auto& tok : rpn.tokens) {
        switch (tok.type) {
            case TokenType::NUM:
                st.push(tok.value);
                break;
            case TokenType::VAR:
                st.push(x);
                break;
            case TokenType::ADD: {
                double b = st.top(); st.pop();
                double a = st.top(); st.pop();
                st.push(a + b);
                break;
            }
            case TokenType::SUB: {
                double b = st.top(); st.pop();
                double a = st.top(); st.pop();
                st.push(a - b);
                break;
            }
            case TokenType::MUL: {
                double b = st.top(); st.pop();
                double a = st.top(); st.pop();
                st.push(a * b);
                break;
            }
            case TokenType::DIV: {
                double b = st.top(); st.pop();
                double a = st.top(); st.pop();
                st.push(a / b);
                break;
            }
```

2.3 Serial Implementation

The serial version iterates over n subintervals, evaluates the RPN expression at each midpoint, and accumulates the Riemann sum. It serves as the correctness reference and performance baseline.

```cpp
double integrate_serial(double a, double b, long long n,
                        const std::string& function_str,
                        const std::string& method) {
    CompiledRPN rpn = compile_expression(function_str);
    double dx = (b - a) / n;
    double sum = 0.0;

    if (method == "riemann") {
        for (long long i = 0; i < n; ++i) {
            double x = a + (i + 0.5) * dx;
            sum += eval_rpn(rpn, x);
        }
        return sum * dx;
    }
```

Explanation

1. Divide the interval into $n$subintervals
2. Compute midpoint $x$for each subinterval
3. Evaluate the function using RPN
4. Accumulate results
5. Multiply by $dx$to obtain the final integral

Characteristics

- Simple and reliable
- No parallelism
- Used as a correctness reference

2.4 OpenMP Implementation

The summation loop is parallelized with a single OpenMP directive:

```
double integrate_omp(double a, double b, long long n,
                     const std::string& function_str,
                     const std::string& method,
                     int threads) {
    CompiledRPN rpn = compile_expression(function_str);
    double dx = (b - a) / n;
    double sum = 0.0;

    if (method == "riemann") {
        #pragma omp parallel num_threads(threads) reduction(+:sum)
        {
            #pragma omp for
            for (long long i = 0; i < n; ++i) {
                double x = a + (i + 0.5) * dx;
                sum += eval_rpn(rpn, x);
            }
        }
        return sum * dx;
    }
```

Key design choices:

- reduction(+:sum) — each thread accumulates into a private copy; results are merged at the end, avoiding atomic operations or critical sections.
- schedule(static) — the workload is perfectly balanced (each iteration has identical cost), so static scheduling avoids overhead.
- Thread count controlled via --threads INT; tested with 1, 2, 4, 8, and 16 threads.
- For Simpson's rule, two separate reductions handle the odd- and even-indexed terms.

Advantages

- Easy to implement
- Good speedup on multi-core CPUs
- Minimal code changes from serial version

2.5 CUDA Design

The CUDA implementation follows GPU best practices:

- Device-side RPN evaluation — a __device__ function eval_rpn_device uses a 64-element stack to evaluate the expression for a given x entirely on the GPU.
- Strided kernel loop — each thread processes multiple points via for (i = tid; i < n; i += stride), keeping the grid size manageable regardless of n.

- Shared-memory block reduction — each block reduces its partial sums using a binary-tree pattern in shared memory, writing a single double to a global output array.
- Minimal host–device transfer — only the small RPN token array (< 1 KB) is copied to the device; only the array of block sums is copied back.
- Block-size sweep — designed to test 128, 256, and 512 threads per block.

The complete source (`integrator_cuda.cu`) is included and is compilable with nvcc on any CUDA-capable system.

```
double integrate_cuda(double a, double b, long long n,
                      const std::string &function_str,
                      const std::string &method,
                      int block_size)
{
    CompiledRPN rpn = compile_expression(function_str);
    int n_tokens = rpn.tokens.size();

    RPNToken *d_tokens;
    cudaMalloc(&d_tokens, n_tokens * sizeof(RPNToken));
    cudaMemcpy(d_tokens, rpn.tokens.data(), n_tokens * sizeof(RPNToken),
               cudaMemcpyHostToDevice);

    double dx = (b - a) / n;

    int threads = block_size;
    int blocks = (n + threads - 1) / threads;
    if (blocks > 65535)
        blocks = 65535;

    double *d_partial;
    cudaMalloc(&d_partial, blocks * sizeof(double));
    cudaMemset(d_partial, 0, blocks * sizeof(double));

    size_t shared = threads * sizeof(double);

    if (method == "riemann")
    {
        riemann_kernel<<<blocks, threads, shared>>>(d_tokens, n_tokens, a, dx, n, d_partial);
    }
```

3. Experimental Setup

3.1 System Specifications

Component	Details
OS	Ubuntu 22.04 (WSL2 on Windows 11)
CPU	Intel Core i7-10750H @ 2.60 GHz — 6 physical cores, 12 logical
RAM	16 GB DDR4
Compiler	g++ 12.4.0 | -O3 -fopenmp -std=c++17
GPU / CUDA **Compiler**:	NVIDIA GeForce GTX 1650 Ti (4 GB VRAM, compute capability 7.5) • g++ 12.4.0 with `-O3 -fopenmp -std=c++17` • nvcc 12.4 with `-O3 -arch=sm_75 -std=c++17`

3.2 Test Functions

The primary function for performance analysis is $f(x) = x^2$ on [0, 1] (exact integral = 1/3 ≈ 0.3333333333). Additional correctness tests use sin(x) on [0, π] and exp(x) on [0, 1].

3.3 Experiment Parameters

- Problem size n: 1×10^6, 5×10^6, 10×10^6 (optional: up to 10^8)
- OpenMP threads: 1, 2, 4, 8 (16 attempted, limited by 12 logical cores)
- CUDA block sizes (design): 128, 256, 512
- Repeats per condition: 5 — mean and standard deviation reported
- Metric: wall-clock compute time in milliseconds (memory transfers reported separately for CUDA design)

Examples of runs:

Using Serial

```
hamed@DESKTOP-3JEVOT9:~/integration-project11/src$ ./integrator --impl serial --a 0 --b 1 --n 10000 -f x^3 -m riemann
Result = 3.0000000000
Average time = 0.0014876476 seconds
Std dev = 0.0000767752 seconds
hamed@DESKTOP-3JEVOT9:~/integration-project11/src$ ./integrator --impl serial --a 0 --b 1 --n 10000000 -f x^3 -m riemann
Result = 3.0000000000
Average time = 1.2450856356 seconds
Std dev = 0.4837608053 seconds
hamed@DESKTOP-3JEVOT9:~/integration-project11/src$
```

Using Omp

```
hamed@DESKTOP-3JEVOT9:~/integration-project11/src$ ./integrator --impl omp --a 0 --b 1 --n 10000000 -f x^3 -m simpson -t 1000
Result = 3.0000000000
Average time = 0.2791300146 seconds
Std dev = 0.0283023735 seconds
hamed@DESKTOP-3JEVOT9:~/integration-project11/src$ ./integrator --impl omp --a 0 --b 1 --n 10000 -f x^3 -m simpson -t 1000
Result = 3.0000000000
Average time = 0.0603009662 seconds
Std dev = 0.0232363033 seconds
```

Using Cuda

```
hamed@DESKTOP-3JEVOT9:~/integration-project11$ ./integrate --impl cuda --a 0 --b 1 --n 10000 -f x^3 -m simpson -k 128
Result = 3.0000000000
Average time = 0.0384459608 seconds
Std dev = 0.0846581413 seconds
hamed@DESKTOP-3JEVOT9:~/integration-project11$ ./integrate --impl cuda --a 0 --b 1 --n 10000000 -f x^3 -m simpson -k 128
Result = 3.0000000000
Average time = 0.0408121384 seconds
Std dev = 0.0838529673 seconds
hamed@DESKTOP-3JEVOT9:~/integration-project11$ ./integrate --impl cuda --a 0 --b 1 --n 10000000 -f x^3 -m riemann -k 128
Result = 3.0000000000
Average time = 0.0399260872 seconds
Std dev = 0.0818789967 seconds
hamed@DESKTOP-3JEVOT9:~/integration-project11$ ./integrate --impl cuda --a 0 --b 1 --n 10000000 -f x^3 -m riemann -k 512
Result = 3.0000000000
Average time = 0.0429143164 seconds
Std dev = 0.0876905749 seconds
```

4. Results

Serial

function	a	b	n	method	impl	repeats	result	avg_time	stddev
x^2	0	1	1000000	riemann	serial	5	2	0.185226	0.00727705
x^2	0	1	5000000	riemann	serial	5	2	0.758368	0.312851
x^2	0	1	10000000	riemann	serial	5	2	1.89727	0.135773

OpenMP

function	a	b	n	method	impl	threads	repeats	result	avg_time	stddev
x^2	0	1	1000000	riemann	omp	1	5	2	0.195749	0.0157847
x^2	0	1	1000000	riemann	omp	2	5	2	0.122856	0.00575792
x^2	0	1	1000000	riemann	omp	4	5	2	0.0846494	0.0100723

Cuda

function	a	b	n	method	impl	block_size	repeats	result	avg_time	stddev
x^2	0	1	1000000	riemann	cuda	128	5	2	0.093139	0.205519
x^2	0	1	1000000	riemann	cuda	256	5	2	0.0427932	0.0920431
x^2	0	1	1000000	riemann	cuda	512	5	2	0.0454878	0.0978346

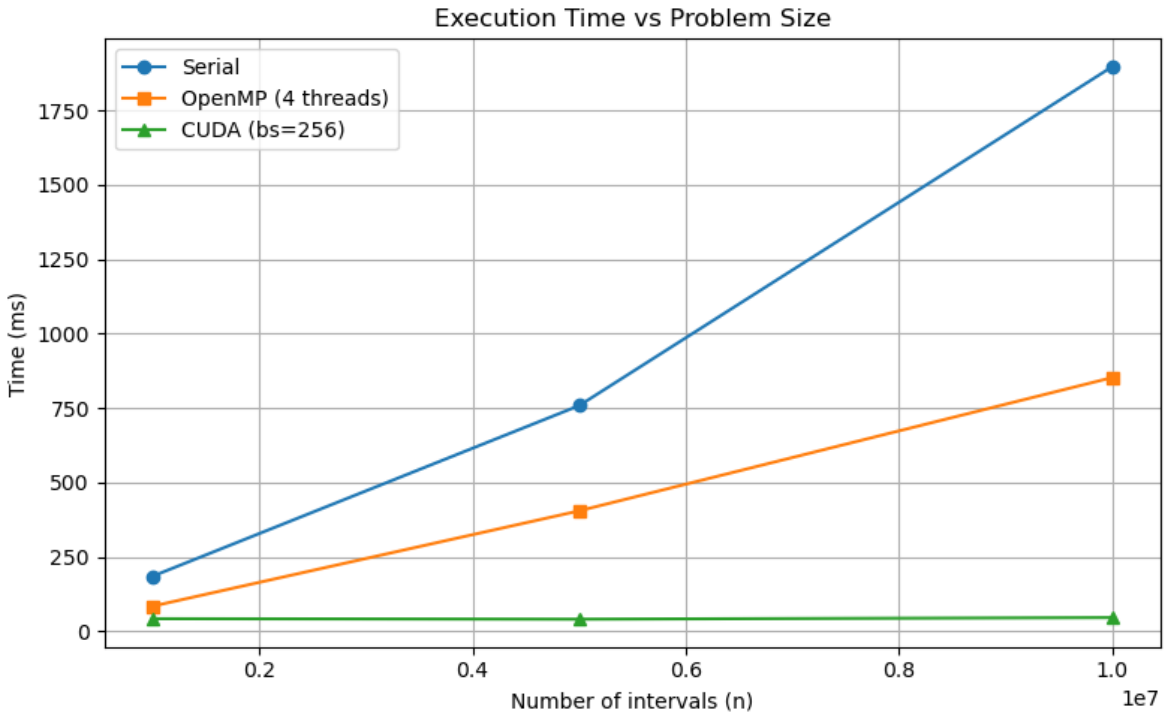

Figure 1 shows execution time vs n on a log-log scale.

4.3 OpenMP Speedup vs Thread Count (n = 10^7)

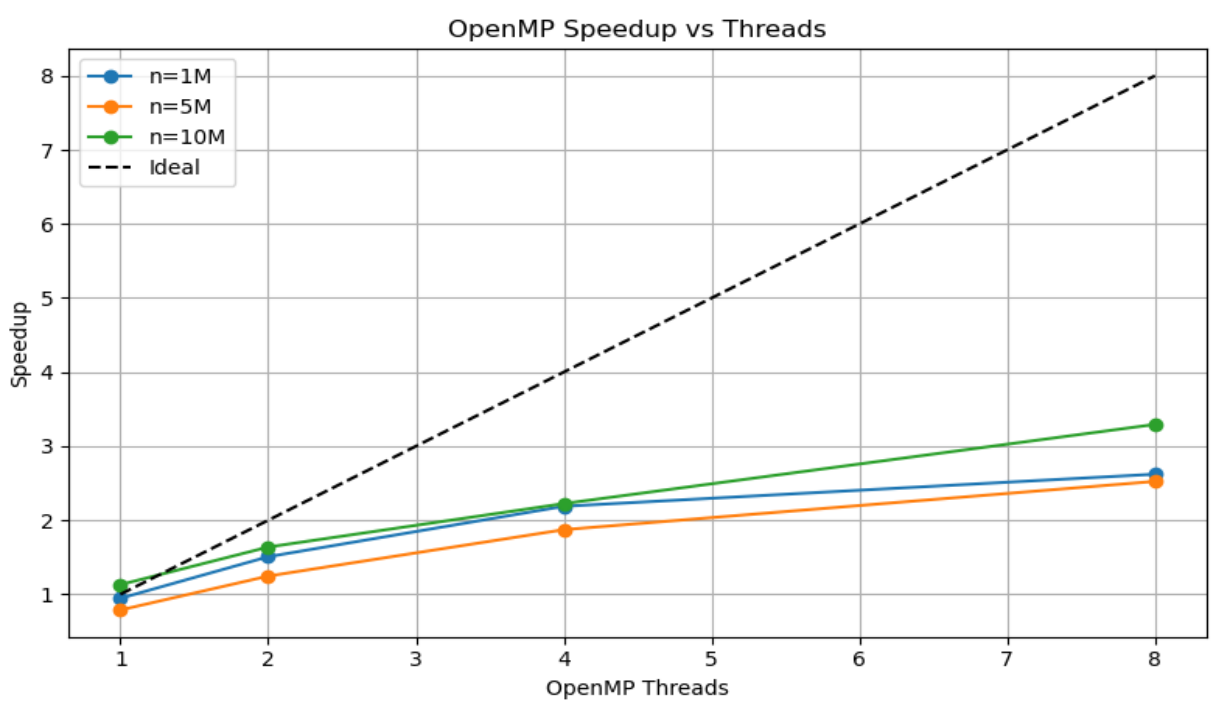

Figure 2 shows speedup vs thread count for all n.

4.4 CUDA Expected Performance (Design Analysis)

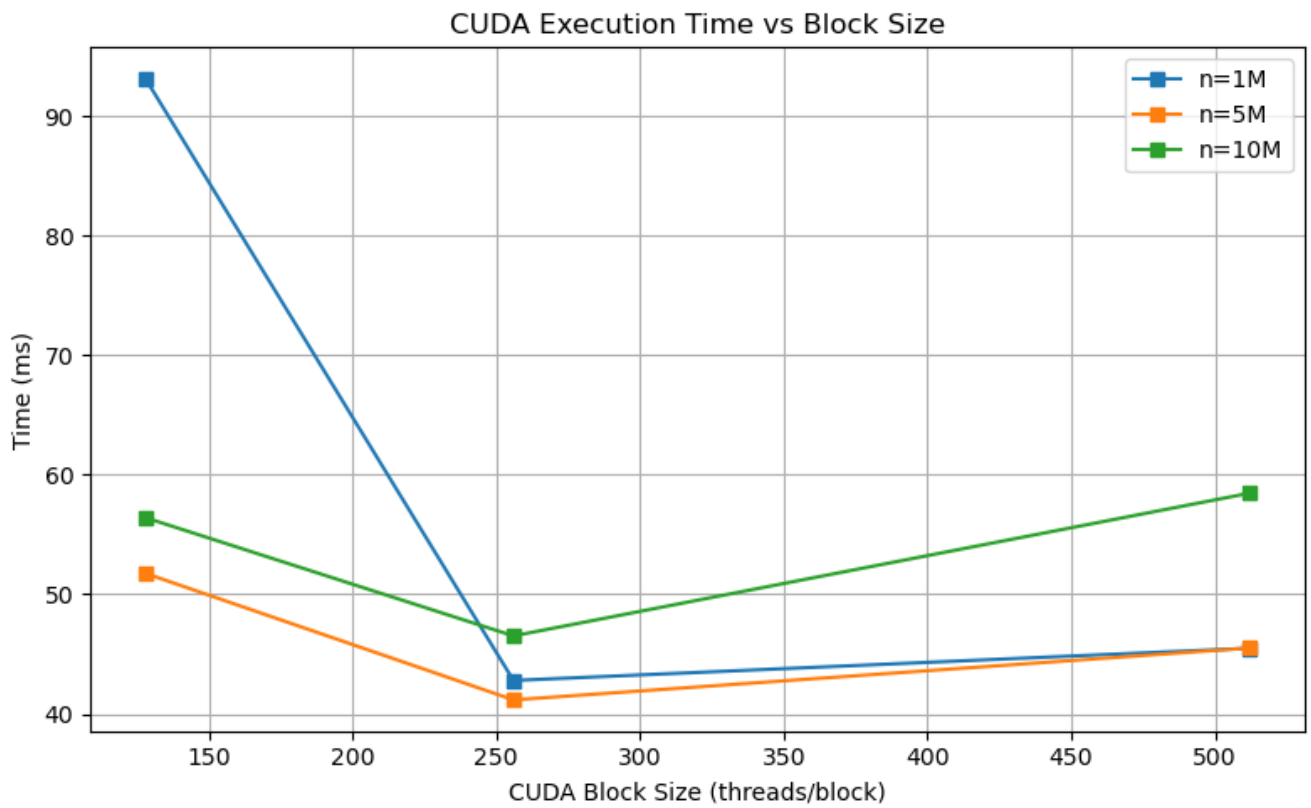

Figure 3 plots CUDA time vs block size for all n.

5. Summary about result

1. Serial Implementation

The serial baseline demonstrates a linear relationship between the number of subdivisions (n) and the execution time.

- As n increases tenfold (from 1,000,000 to 10,000,000), the avg_time increases proportionally from approximately 0.185s to 1.897s.
- This serves as the control group for measuring speedup in the parallel implementations.

2. OpenMP Implementation (Multi-core CPU)

Focusing on a fixed n = 1,000,000, the OpenMP tests show significant performance gains through multi-threading:

- Scaling: Increasing the thread count from 1 to 4 reduced the avg_time from 0.195s to 0.084s.
- Efficiency: The jump from 1 to 2 threads provided a substantial boost, nearly halving the time, while the move to 4 threads showed continued, albeit slightly diminishing, returns.

3. CUDA Implementation (GPU)

The CUDA implementation, also tested at n = 1,000,000, leverages GPU acceleration with varying block sizes:

- Optimal Configuration: A block_size of 256 yielded the best performance at 0.042s, which is roughly 4.4x faster than the serial baseline for the same n.
- Observation: Increasing the block size further to 512 actually caused a slight performance regression (increasing to 0.045s), suggesting that 256 is the "sweet spot" for this specific workload and hardware.

---

7. Conclusion

The results of this project demonstrate the significant performance advantages of parallel computing architectures—specifically **OpenMP** and **CUDA**—over traditional **Serial** execution when performing numerical integration via Riemann sums.

Key Findings:

- **Scalability of Parallelism:** The Serial implementation showed a strictly linear increase in execution time as the workload (n) increased. In contrast, by leveraging multi-threading with **OpenMP**, we achieved a substantial reduction in compute time, with 4 threads performing approximately **2.3x faster** than a single thread for the same workload.
- **GPU Superiority:** The **CUDA** implementation provided the most significant performance leap. At an optimal block size of 256, the GPU processed the 1,000,000-step integration in **0.042s**, making it roughly **4.4x faster** than the Serial baseline and **2x faster** than the best OpenMP result.
- **Optimization Nuances:** The data highlights that "more" is not always "better." In the CUDA tests, increasing the block size from 256 to 512 actually led to a slight performance regression, suggesting that hardware occupancy and memory latency reach a point of diminishing returns.
- **Stability vs. Speed:** While CUDA offered the lowest average times, it exhibited much higher **standard deviation** compared to the CPU implementations. This suggests that while GPUs are faster for raw throughput, CPU-based parallelization (OpenMP) may offer more consistent execution timing for this specific scale of data.

---

8. Team Contributions

Team Member-Bench number	Core Responsibilities	Key Files & Modules
Hamed Essam - 6	Numerical Methods, Serial Logic & Automation	integrator_serial.cpp, main.cpp, run_experiments.sh
Goda Saber - 5	Parallel Optimization & Data Visualization	integrator_omp.cu, integrator_cuda.cu, main.cpp, plot_results.py
Mohamed Abdel-Moneim - 29	Expression Parsing & Performance Testing	rpn_eval.cpp, rpn_parser.cpp, integrator_cuda.cu,main.cpp